STUDY OF DUPLICATIONS IN LINUX KERNEL SUBSYSTEMS: ANALYSIS, REFACTORING, AND CONTRIBUTIONS

Guilherme Ivo Bozi, Gabriel Sousa Santos de Almeida,

Natália Salvino André, Paulo Meirelles

Institute of Mathematics, Statistics and Computer Science/University of São Paulo

{guilherme.bozi, gabrielsousa230}@usp.br

Objectives

The Linux kernel is one of the most relevant free software projects, showing continuous growth in both size and number of contributors[6]. In this context, the application of good software engineering practices becomes important to ensure code quality and maintainability. Therefore, this work aims to identify and mitigate code duplications in the Linux kernel with the support of `ArKanjo`[2], a code duplication detection tool, as well as to analyze and select cases for the application of refactoring techniques, prioritizing maintainability criteria[1]. Based on this analysis, it seeks to propose and submit patches (contributions) containing refactorings to the Linux kernel community. In addition, it aims to document the process and interactions with the community through participant observation[5, 4], systematizing the lessons learned and reflecting on the socio-technical aspects of the contribution process.

Materials and Methods

The `ArKanjo` has a two-stage architecture (Figure 1), updated in this work from the initial model[2]. The Preprocessor processes the code base and generates representations of the analyzed units, while the Query Responder allows queries to be performed without reprocessing the code. The tool now uses `Tree-sitter`, expanding support for different programming languages and enabling AST-based analyses. It also incorporated a new similarity method based on code embeddings, along with improvements to command extensibility and support for structured and custom outputs.

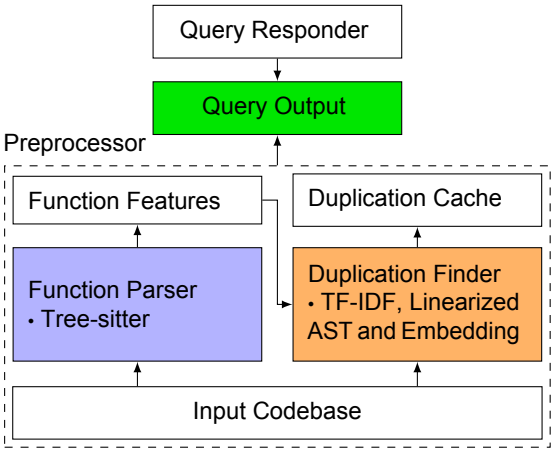


Figure 1: Architecture diagram showing the components of the tool

To evaluate the applicability of `ArKanjo` in Linux kernel development, an ethnographic and multi-method study was conducted, divided into two phases, following the process from the identification of duplications to the implementation and submission of potential contributions.

In **Phase 1**, `ArKanjo` was used to identify duplications with 90% and 100% similarity, select the

most promising cases, and analyze their refactoring potential. In **Phase 2**, the patches were implemented and submitted to the maintainers, including revisions and adjustments when requested. The process was applied to the AMDGPU and IIO subsystems in two contexts: in the **Participant Observation**, the authors carried out the contributions, seeking to understand the difficulties involved in the development process and document interactions with maintainers. In the **Study with Participants**, students from the MAC0470/5856 course (USP, 2026) acted as contributors, either carrying out Phase 1 or using duplications previously identified by the authors and focusing on Phase 2.

Results

A total of 17 patches were submitted to the AMDGPU and IIO subsystems, of which 8 were accepted, 4 remain under review, and 5 were not accepted, resulting in a net removal of 1,096 lines of code. Interactions with maintainers indicated that the application of the DRY (*Don't Repeat Yourself*) principle[3] in the kernel is contextual, taking into account aspects such as isolation between hardware generations, simplicity, and readability.

Table 1: Summary of refactoring patch submissions, grouped by status.

Status	Patches	Diff
Accepted	8	`+1541/-2637`
Under review	4	`+140/-248`
Not accepted	5	`+388/-584`

Conclusions

The results show that the process of transforming duplications into contributions goes beyond automatic identification, highlighting the importance of context analysis and interaction with the community. In this regard, `ArKanjo` proved useful for supporting the exploration of an extensive code base and the identification of contribution opportunities. These results demonstrate its potential as a tool to support the participation of new contributors and the discovery of opportunities for improvement in the Linux kernel.

The authors declare no conflicts of interest. **Guilherme** contributed by submitting 4 patches and by preparing this abstract. **Gabriel** contributed by submitting 2 patches and by preparing this abstract. **Natália** contributed one patch and assisted in reviewing this text. **Paulo** provided guidance on the conception and planning of the study. All authors approved the final version of this abstract.

This study was financed, in part, by the São Paulo Research Foundation (FAPESP), Brasil. Process Number #2025/22416-1.

References

[1] E. L. G. Alves, P. D. L. Machado, T. Massoni, and S. T. C. Santos. A Refactoring-Based Approach for Test Case Selection and Prioritization. In *Proc. 8th AST*, 2013.

[2] L. Arcanjo, D. Tadokoro, and P. Meirelles. ArKanjo: a tool for detecting function-level Code Duplication in the Linux Kernel. In *DebConf25*. IRISA, 2025.

[3] L. Arcanjo, D. Tadokoro, M. Spessoto, R. Passos, and P. Meirelles. When Do You Repeat Yourself? Voices from the Trenches of Linux Kernel Maintainers on Code Duplication. In *TechDebt'26*. ACM, 2026.

[4] C. Robson and K. McCartan. *Real World Research: A Resource for Users of Social Research Methods in Applied Settings*. John Wiley & Sons, 4 edition, 2016.

[5] C. B. Seaman. Qualitative Methods. In *Guide to Advanced Empirical Software Engineering*, pages 35–62. Springer, 2008.

[6] Willie. Linux kernel contributors and lines of code statistics 2026, 2026.